

INTERVIEW NOTES

For Saurabh — revise the whole project from this one document

Written so that after the FastAPI migration, you can explain any part of this system on a whiteboard without opening the code. Each module: why it exists, the full request lifecycle through it, the decisions you should be able to defend, and the questions an interviewer will actually ask.

Module 1: Authentication & Authorization

Why it exists: Four roles, strictly hierarchical, campus-scoped. You need to know *who* is calling and *what campus/outlet* they belong to on every request, statelessly (no server-side session), across a system where different roles were onboarded through completely different flows (seeded, applied-and-approved, applied-and-approved-with-documents, self-registered-with-domain-check).

Request lifecycle (login): POST /login → look up user by email → BCrypt-verify password → check accountStatus (reject PENDING_VERIFICATION and anything non-ACTIVE with 403) → issue JWT (sub=email, role claim, HS256, 24h expiry) → client stores it, attaches Authorization: Bearer on every future request → a filter/dependency decodes and validates the token on every request → sets "who is this" in request context → route-level role check compares against an allow-list.

Design decisions you should be able to defend:

- *JWT carries only email + role, not user id or campus id.* Trade-off: smaller token, simpler claim surface, but every request that needs "which campus" re-queries the DB. This is a deliberate statelessness-over-round-trips choice, not an oversight.
- *No refresh tokens, no revocation.* A stolen token works until natural expiry (24h). Accepted risk for a project at this stage — the honest answer in an interview is "yes, this is a known limitation, here's how I'd add refresh tokens if this went to production" (short-lived access token + long-lived refresh token in an httpOnly cookie + a revocation table), not pretending it's not a gap.
- *Two-step registration* (register → email OTP → verify → login) exists specifically to guarantee every account has a real, owned email before it can do anything — this matters because email domain is the *entire* campus-membership proof for students.

Interview questions:

- "Why JWT over sessions here?" → Stateless, horizontally scalable without sticky sessions or a shared session store; trade-off is you lose easy revocation.
- "How do you prevent a student from calling an admin endpoint?" → Role claim in the token, checked against a per-route allow-list before the handler runs — not inside business logic, so it can't be forgotten per-endpoint.
- "What happens if the JWT secret leaks?" → Every token becomes forgeable; rotation requires re-issuing all tokens (everyone logged out) since there's no per-user secret.
- **Follow-up they might push on:** "How would you add logout?" → You can't truly invalidate a stateless JWT without a server-side check; either maintain a short-lived denylist (defeats some statelessness benefit) or switch to short-lived tokens + refresh rotation.

Whiteboard-ready explanation: draw Client → [JWT in Authorization header] → Auth check (decode + validate) → Role check (allow-list) → Handler → re-fetch full User row by email if needed → response.

Module 2: Multi-Tenant Roles & Onboarding

Why it exists: This isn't one platform — it's N independent campus platforms sharing infrastructure. Each campus needs its own admin, its own outlets, and strict boundaries so an ADMIN from Campus A can never see or touch Campus B's data.

Request lifecycle (campus + outlet onboarding):

1. Prospective admin submits an application with campus name/domain/their designation → OTP-verifies their email → SUPERADMIN reviews → approve creates a Campus row + an ADMIN user in one action.
2. Prospective outlet manager searches for their campus, submits an application with business documents → an automated format-validator checks document-shaped fields (GST/FSSAI/IFSC patterns — **format only, not a live government API call**) and produces a verification report → the campus's ADMIN reviews the report and approves/rejects → approval creates an Outlet (status PENDING_LAUNCH) + a MANAGER user.
3. Manager configures menu/slots/bank details, then "launches" the outlet (status → ACTIVE), which is the moment students can see it.

Decisions to defend:

- *Document verification is local format validation, not a real third-party API call.* Be upfront about this if asked — it's a legitimate simplification for a student project

(real GST/FSSAI verification APIs cost money and require business registration to even access), and the architecture (a distinct `VerificationReport` entity/step) is specifically designed so a *real* verification API can be dropped in later without changing the approval workflow around it.

- *Both admin and manager applications are capped at 3 attempts if rejected* — a simple anti-abuse measure, not because of any technical constraint.

Interview questions:

- "How do you enforce campus isolation?" → Every campus-scoped entity has a `campus_id` foreign key; every query for campus-scoped data filters by the requesting user's own campus, resolved server-side from their user row — never trusted from client input.
- "Why two different onboarding flows (admin vs manager)?" → Different trust levels and different verification needs: an admin's legitimacy is about *institutional* identity (does this email domain really belong to this college), a manager's legitimacy is about *business* identity (is this a real, registered food outlet with real bank details for payouts).

Module 3: Menu & Pickup Slots

Why it exists: This is the core scarcity-management mechanism that makes the whole "reduce queues" pitch real. Without slot capacity limits, "ordering ahead" doesn't actually spread out pickup demand.

Request lifecycle: Manager opens outlet in the morning → creates first slot with a max-orders limit → each slot auto-expires after its hour window, and the next slot inherits the same limit unless the manager changes it → students browsing see remaining capacity and time-left per slot → once a slot hits its cap, students see it as unavailable and get pushed toward another slot/outlet (ML recommendation angle).

Decisions to defend:

- *Optimistic locking with exactly one retry* on slot capacity, not a DB-level pessimistic lock or an app-level mutex. Why: pessimistic locks serialize all writers to a slot (bad for throughput at peak ordering time); optimistic locking lets concurrent bookings proceed and only pays a retry cost on an actual conflict, which is rare relative to total bookings.
- *Why exactly one retry, not infinite retries or zero?* Zero retries means a benign race (two students booking the literal last two slots simultaneously) unnecessarily fails one of them even though there was room a moment before their conflicting write; infinite retries risks a request hanging under sustained

contention. One retry is the practical middle ground for this traffic pattern (bursty around meal times, not sustained high-contention).

Interview questions:

- "How do you prevent slot overbooking under concurrent requests?" → Version column, optimistic check-and-increment, retry once on conflict, fail with a clear "slot full" error on the second conflict.
 - "Counter orders can exceed slot capacity — isn't that a bug?" → No, deliberate: capacity limits protect *student* experience/UX predictability; a manager serving a walk-in in person has already made the real-world capacity decision themselves, so the system defers to their judgment rather than blocking a sale.
-

Module 4: Orders — the core transaction

Why it exists: This is where every other module converges: auth, slots, menu, ML, payments, notifications all meet in one request.

Request lifecycle (student places an online order): Auth check → re-fetch student row → check no unpaid penalty is blocking them → check outlet exists/active/within operating hours → check daily order cap (3/outlet/day) → check slot exists and has capacity → validate each ordered item's price/availability, compute total → call ML service for wait-time estimate (fallback: 20 min if ML is down) → call ML service for no-show risk (fallback: 0.5; if high risk, queue a manager notification) → save the order → (COD only) generate the deterministic pickup OTP and save again → save order-item rows → increment slot's current-order count with the optimistic-lock-retry-once pattern → return the order to the student. If ONLINE, payment (and OTP generation) happens in a *separate* follow-up call after Razorpay checkout succeeds, not in this same request.

Decisions to defend:

- *Deterministic, HMAC-derived pickup OTP* (HMAC(orderId : minute-bucket)), not random-and-stored. Why: no uniqueness check/DB round-trip needed to generate one; it's cryptographically tied to the specific order so it can't be reused or guessed without the server secret, and verification is a trivial stored-value string compare — no re-derivation, no clock-skew edge cases at *verify* time (only at *generate* time, which happens once).
- *Two independent OTP systems* (email-verify vs. pickup) deliberately kept separate — different purpose, different lifecycle, different expiry semantics; conflating them would be a security smell (using an account-proof OTP as a physical-handoff proof, or vice versa).

- *Status machine is PLACED → PREPARING → READY → PICKED*, with CANCELLED/EXPIRED as terminal side-branches — deliberately no ACCEPTED step (an early design doc had one; the shipped implementation collapsed it because in practice the manager accepting and starting prep are the same action for this use case).
- *ML calls are synchronous, in the checkout hot path*, guarded by fallback constants and a timeout. This is a real, known latency trade-off: the wait-time/risk numbers are valuable enough at order-creation time to justify the added latency, but the fallback pattern means the system never actually depends on ML availability for correctness — only for quality of the estimate.

Interview questions:

- "Why not fire the ML calls asynchronously?" → You need the wait-time estimate *in the response* the student sees immediately after ordering — could parallelize the two ML calls with each other (legitimate async improvement), but can't move them fully out-of-band without changing what the student sees at checkout.
- "What happens if two students hit the last slot spot at the same moment?" → Covered in Module 3 — optimistic lock, one retry, one of them gets a clean "slot full" response instead of an overbooked slot.
- "Why generate the OTP with HMAC instead of just random.randint(1000,9999)?" → Determinism means zero collision-checking cost and the value is cryptographically bound to that specific order — a random OTP would need a uniqueness constraint and a regeneration loop on collision, plus no way to "recompute and check" if you ever lost the stored value (though in practice you never need to, since it's derived once and stored, exactly like a random one would be — the real win is skipping the uniqueness dance, not avoiding storage).

Module 5: Penalty System — the demand-weighted no-show fine

Why it exists: The core innovation pitch. A flat no-show fine is either too harsh for high-demand items (outlet resells instantly, no real loss) or too lenient for low-demand items (outlet eats a real loss). Weighting the fine by actual historical demand for that item at that time window makes the penalty proportional to actual risk.

Request lifecycle (scheduled job, every 5 minutes): Find orders past expiresAt that are still PLACED/PREPARING/READY. For PLACED/PREPARING ones: mark EXPIRED, **charge nothing** — the outlet never finished preparing, so the student isn't at fault (this is the CRITICAL FIX rule — there was a prior bug where these were incorrectly penalized). For READY-but-uncollected ones: mark EXPIRED, compute a demand score (ask the ML service first; if unavailable, fall back to a rule-based windowed SQL query

counting no-shows for the same item within a ± 30 -minute window of the same time-of-day over the prior 10 days), calculate a demand-weighted penalty from that score, add it to the student's pending penalty balance, increment their no-show count; after the 3rd no-show, flag the account WARNING (blocks new orders until penalties are paid, unless an admin manually lifts it for a documented reason).

Decisions to defend:

- *Why the "no penalty unless READY" rule matters so much it's a CRITICAL FIX comment in the code:* it encodes a real correctness property — you should never financially punish someone for a failure that wasn't their fault. Any interviewer probing "tell me about a bug you fixed" question maps perfectly here: describe the prior incorrect behavior, why it was wrong, and how the fix works.
- *Two structurally different demand-score computations* (ML vs. rule-based fallback) coexisting on purpose — resilience over elegance. The honest trade-off to state: the *same* no-show event can be penalized slightly differently depending on whether the ML service happened to be up at that exact moment, which is a known, accepted non-determinism, not a hidden bug.
- *Fixed 30-day flat penalty before the ML model has data* — you can't compute a meaningful demand score with zero historical data, so the system bootstraps with a flat rate and switches to the ML-informed formula once 10 days of order history exists. This is a good "cold start problem" answer if asked about ML system design generally.

Interview questions:

- "How would this system behave on day one with zero data?" → Flat penalty for the first ~30 days (or until 10 days of relevant order history accumulates for the fallback query), demand-weighting only kicks in once there's enough signal.
- "Why 3 no-shows before suspension, and why 10 days?" → Judgment calls with no hard technical constraint — reasonable defaults chosen to balance deterrence against not permanently punishing an occasional emergency; worth being honest that these are tunable business parameters, not derived from a formula.

Module 6: Payments & Payouts

Why it exists: Two very different payment flows live here — a synchronous student-facing checkout (Razorpay Orders API) and an asynchronous platform-to-outlet weekly settlement (Razorpay X payouts) — plus the one truly external-facing, unauthenticated-by-design endpoint in the whole system.

Request lifecycle (online payment): Student initiates → backend creates a Razorpay order via the SDK → frontend opens Razorpay Checkout with that order id → on success, frontend posts the payment id/order id/signature to /verify/order → backend recomputes the HMAC signature over those exact values using the Razorpay secret and compares — only on a match does it mark the order PAID and generate the pickup OTP.

Request lifecycle (weekly payout, scheduled Sunday 2 AM): Aggregate the past week's orders per outlet, split ONLINE vs. CASH revenue, deduct a 5% platform commission from the ONLINE portion only, initiate a bank transfer via Razorpay X's REST API (no official SDK support for this product, hence raw authenticated HTTP calls instead of the SDK used for regular payments) — real or simulated depending on a feature flag, useful for keeping this free/testable during development.

Decisions to defend:

- *Why commission only on the ONLINE portion, not CASH?* Cash never passes through the platform's account — there's nothing to deduct from, since the platform never held that money. This is a real, defensible business-logic reason, not an oversight.
- *Why a raw HTTP client for payouts instead of the SDK used for regular payments?* The official razorpay-java/equivalent SDK doesn't cover the Razorpay X payouts product at all — a genuine SDK gap, not a style choice.
- *The webhook endpoint is the only one in the system not authenticated by JWT — it's server-to-server (Razorpay calling you), so it's authenticated by an HMAC signature over the raw request body instead.* Good interview point on "not everything needs the same auth mechanism — auth should match who's actually calling."

Interview questions:

- "Walk me through preventing a forged payment success." → Client never gets to say "trust me, it succeeded" — the backend independently recomputes the HMAC over the payment/order id pair using a secret only the backend and Razorpay know, and only a match flips the order to PAID.
- "Why is /payments/webhook/razorpay public in the security config, and is that a hole?" → Not a hole because "public" here means "no JWT required," not "no verification" — its own HMAC check is the real gate. Worth stating clearly if asked, since it's the kind of thing that *looks* wrong in a config file skim but is correct by design.

Module 7: Notifications & Live Order Tracking (SSE)

Why it exists: Students need live status updates without polling, and this needs to work across multiple backend instances if the app ever scales horizontally.

Request lifecycle: Manager transitions an order's status → service writes the DB row → publishes a message on a Redis pub/sub channel → every backend instance subscribed to that channel pushes the update to any locally-held SSE connection for that order → also writes a Notification DB row for the persistent notification feed.

Decisions to defend:

- *Redis pub/sub, not a direct in-process push*, specifically so this works correctly if the backend is ever deployed as more than one instance — a single-instance in-memory map would silently miss updates for connections held by a *different* instance.
- *SSE, not WebSockets* — order status updates are one-directional (server → client only), so SSE's simpler HTTP-based model is a legitimate, deliberate fit rather than a missed opportunity to use WebSockets.

Interview questions:

- "Why SSE over WebSockets here?" → The communication is one-way; SSE avoids the extra complexity of a bidirectional protocol you don't need, and works over plain HTTP (simpler infra, works through more proxies by default).
- "What happens if a student opens the tracking page on two devices at once?" → Known limitation: the emitter registry currently keys by order id alone, so a second connection for the same order can replace/orphan the first on the same instance — a good "here's a known limitation and how I'd fix it" talking point (key by order id *and* a connection/session id instead).

Module 8: The ML Microservice

Why it exists: Genuinely differentiated features (personalized recommendations, dynamic wait-time prediction, demand-weighted penalties, no-show risk, slot-demand forecasting) that a plain CRUD backend can't do, kept as a separate deployable so model training/retraining never competes with or destabilizes the transactional API.

Architecture point to nail in an interview: it's a **shared-database, not shared-API**, integration for training data — the ML service reads the same Postgres tables directly for building its training sets, but is called over HTTP for actual inference. This is explicitly a trade-off: simpler than building a data-export pipeline or event stream between services, at the cost of tight coupling — any schema change in the main backend can silently break the ML service since there's no API/contract boundary protecting it. Good, honest answer if asked "what would you do differently at scale?": introduce a proper

data-export/event pipeline (e.g., CDC or a scheduled ETL) instead of direct cross-service table access.

Interview questions:

- "Why is the ML service reading the database directly instead of getting data from the main API?" → Faster to build for a single-developer project timeline, acceptable coupling risk at this scale; the honest cost is no schema-change safety net between the two services.
- "What happens to the user experience if the ML service goes down entirely?" → Nothing breaks — every call site has a typed fallback value baked in, so functionality degrades to "less personalized/less accurate estimates," never to an error.

General System-Design Talking Points (any module)

- **Graceful degradation as a first-class design pattern**, not an afterthought — the ML fallback contract is the clearest example: correctness never depends on an optional subsystem's availability.
- **Multi-tenancy via a foreign key + server-side filtering**, not separate databases/schemas per campus — simpler ops, acceptable isolation for this scale, a real trade-off to name if asked "how would this scale to 500 campuses?" (that's the point where per-tenant schema or DB sharding conversations start).
- **Why migrate Spring Boot → FastAPI at all?** Be honest and specific if asked: unifying on one language across both backend services simplifies solo-developer maintenance, async-native FastAPI is a better fit for a service that's I/O-bound on ML calls and DB queries, and it removes a second build toolchain (Maven) from a one-person project's operational surface. It is *not* "FastAPI is objectively better than Spring" — name the actual trade-offs (Spring's mature ecosystem/tooling vs. Python's faster iteration for a small team) rather than a one-sided answer.
- **What you'd add for real production deployment:** refresh tokens + revocation, Alembic migrations, real GST/FSSAI verification API integration, paid Razorpay keys, automated test coverage (there was none before this migration), an actual data pipeline instead of shared-DB ML access, per-connection (not per-order) SSE keys.